



Programming

Files

Vassilis Markos, Mediterranean College

Week 07

Contents



- 1 Last Time's Exercises
- 2 Files + Python
- 3 Fun Time!

Last Time's Exercises

In-class Exercise #001



In today's materials, under the following link:

`../labs/Programming_Lab_02.pdf`

you can find the second part of a Lab series for this course. Follow the instructions found therein to start working on it!

Files + Python

Creating A Simple File



Consider the following file (`source/students.txt`):

```
1 Name , Age , Grade
2 Alice , 20 , A
3 Bob , 22 , B
4 Charlie , 19 , C
```

- Our task is to parse all those precious data stored in that file and create a Python dictionary with those data — for future use.
- How can we even do that?

Reading A File In Python

We can use something like the following:

```
1 # source/files_001.py
2
3 if __name__ == "__main__":
4     file = open("students.txt", "r") # Open the file
5     students = dict() # Initialise an empty dictionary
6     for line in file.readlines(): # Iterate over the file's lines
7         data = line.split(",")
8         # data is a list containing: [name, age, grade] as *strings*
9         if data[0] != "Name": # In case we read the first line, we just ignore
            it
10             students[data[0]] = (data[1], data[2].strip())
11             # Why do we use .strip()?
12 file.close()
13 print(students)
```

A Typical Output



The above should print something along the following lines:

```
1 {'Alice': ('20', 'A'), 'Bob': ('22', 'B'), 'Charlie': ('19', 'C')}
```

A Typical Output



The above should print something along the following lines:

```
1 {'Alice': ('20', 'A'), 'Bob': ('22', 'B'), 'Charlie': ('19', 'C')}
```

- There is also a case it might have not even run.

A Typical Output



The above should print something along the following lines:

```
1 {'Alice': ('20', 'A'), 'Bob': ('22', 'B'), 'Charlie': ('19', 'C')}
```

- There is also a case it might have not even run.
- If so, what is the console writing?

A Typical Output



The above should print something along the following lines:

```
1 {'Alice': ('20', 'A'), 'Bob': ('22', 'B'), 'Charlie': ('19', 'C')}
```

- There is also a case it might have not even run.
- If so, what is the console writing?
- How can you fix it?

The Currently Working Directory (CWD)

- Depending on your Operating System, you might need to properly inform Python about where to look for files you want to parse.
- You can achieve that typically using the following lines of code (adjust for your case):

```
1 import os
2
3 if __name__ == "__main__":
4     CWD = os.path.abspath(os.path.dirname(__file__))
5     PATH = os.path.join(CWD, "students.txt")
```

`os.path`



In the above we used several functions of the `os` Python module:

- `os` offers OS-specific functionalities for all popular OSs.
- `os.path` is a submodule dedicated to paths.
- `.dirname()` gets the directory name of a certain file.
- `__file__` is a Python variable related to the file from which it is being invoked.
- `.abspath()` retrieves the absolute path, i.e., the path starting from the first point of reference for our system (e.g., C on Windows).
- `.join()` joins a list of paths in an OS-specific way.

Context Managers

- It is really important to close files appropriately, which is typically achieved by using `.close()` on a file object.
- Leaving a file unclosed often has no repercussions but there are cases in which an unclosed file will be inaccessible from other processes.
- So, **do not forget to close files!**
- Python also offers us a nice way to make sure a file is always closed once we are done with it:

```
1 with open("filepath", "r") as file:  
2     # file processing  
3 # here the file is considered closed
```

Reading Structured Files: CSV



Recall the file `source/students.txt`:

```
1 Name, Age, Grade
2 Alice, 20, A
3 Bob, 22, B
4 Charlie, 19, C
```

- This file has a certain structure, where each row contains elements separated by commas.
- Such files are called CSV: Comma Separated Values (files).
- So, isn't there a way to parse them properly?

Reading Structured Files: CSV

```
1 # source/files_003.py
2
3 import os
4 import csv
5
6 if __name__ == "__main__":
7     CWD = os.path.abspath(os.path.dirname(__file__))
8     PATH = os.path.join(CWD, "students.csv")
9     students = dict() # Initialise an empty dictionary
10    with open(PATH, "r") as file: # Open the file
11        reader = csv.reader(file) # Create a csv reader
12        for row in reader: # Iterate over the file's lines
13            # data is a list containing: [name, age, grade] as *strings*
14            if row[0] != "Name": # In case we read the first line, we just
15                ignore it
16                students[row[0]] = (row[1], row[2])
17    print(students)
```

Reading Structured Files: JSON

There are also other structured file formats, like JSON, which resemble Python dictionaries by a lot:

```
1 {  
2   "Alice": [  
3     "20",  
4     "A"  
5   ],  
6   "Bob": [  
7     "22",  
8     "B"  
9   ],  
10  "Charlie": [  
11    "19",  
12    "C"  
13  ]  
14 }
```

Reading Structured Files: JSON



```
1 # source/files_004.py
2
3 import os
4 import json
5
6 if __name__ == "__main__":
7     CWD = os.path.abspath(os.path.dirname(__file__))
8     PATH = os.path.join(CWD, "students.json")
9     with open(PATH, "r") as file: # Open the file
10         students = json.load(file) # Create a csv reader
11     print(students)
```

Reading Structured Files: CSV

```
1 # source/files_005.py
2
3 import os
4 import csv
5 import json
6
7 if __name__ == "__main__":
8     CWD = os.path.abspath(os.path.dirname(__file__))
9     rows = [
10         ["Name", "Age", "Grade"],
11         ["Alice", "12", "A"],
12         ["Bob", "10", "B"],
13         ["Charlie", "14", "C"],
14     ]
15     CSV_PATH = os.path.join(CWD, "students_w.csv")
16     with open(CSV_PATH, "w") as file:
17         writer = csv.writer(file)
18         writer.writerows(rows)
```

Writing Structured Files: JSON



```
1 # source/files_006.py
2
3 import os
4 import csv
5 import json
6
7 if __name__ == "__main__":
8     CWD = os.path.abspath(os.path.dirname(__file__))
9     PATH = os.path.join(CWD, "students.txt")
10    students = dict() # Initialise an empty dictionary
11    with open(PATH, "r") as file: # Open the file
12        reader = csv.reader(file) # Create a csv reader
13        for row in reader: # Iterate over the file's lines
14            # data is a list containing: [name, age, grade] as *strings*
15            if row[0] != "Name": # In case we read the first line, we just ignore it
16                students[row[0]] = (row[1], row[2])
17    JSON_PATH = os.path.join(CWD, "students_w.json")
18    with open(JSON_PATH, "w") as file:
19        json.dump(students, file, indent=2)
```

Wrting Files (In General)



```
1 # source/files_007.py
2
3 import os
4
5 if __name__ == "__main__":
6     CWD = os.path.abspath(os.path.dirname(__file__))
7     PATH = os.path.join(CWD, "a_file.txt")
8     rows = [
9         "This is a file.",
10         "Actually, this is not just a file",
11         "As you might well see on your own...",
12         "...this is a...",
13     ]
14     with open(PATH, "w") as file:
15         for row in rows:
16             file.write(row)
17             file.write("\n")
```

Useful File Modes



- `w`: Write mode (creates a new file or overwrites an existing one).
- `a`: Append mode (adds to the end of an existing file).
- `r`: Read mode (opens an existing file for reading).

There are also the corresponding byte-variants (i.e., for files containing bytes):

- `wb`: Write **bytes** mode (creates a new file or overwrites an existing one).
- `ab`: Append **bytes** mode (adds to the end of an existing file).
- `rb`: Read **bytes** mode (opens an existing file for reading).



Fun Time!

In-class Exercise #001



In today's materials, under the following link:

`../labs/Programming_Lab_03.pdf`

you can find the third part of a Lab series for this course. Follow the instructions found therein to start working on it!

In-class Exercise #002



Start working on all Labs found in today's materials `homework` directory.
To help me asses those files, you can name them as follows:

`task_xxx.py`

where `xxx` is the number of the task. For instance, task 5 file could be named `task_005.py`.

Submit your work via email at: `v.markos@mc-class.gr`

Homework



- In this week's materials, under the `homework` directory, you can find some Python programming Tasks. Complete as many of them as you can (preferably all).
- This is important, since tasks such as those provided with this lecture's materials will most probably be part of your course assessment portfolio. So, take care to solve as many of those tasks as possible!
- Share your work at: `v.markos@mc-class.gr`

Any Questions?

Do not forget to fill in
the questionnaire shown
right!



<https://forms.gle/dKSrmE1VRVWqxBGZA>